



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



HARUURROT: Ride Booking System

A Final Project Submitted to Engr. Godofredo Avena, faculty at the Polytechnic University of the Philippines. In partial fulfillment for requirement in CMPE 103 Object Oriented Programming.

SUBMITTED BY:

Basa, Anika Sophia
Garcia, Stephanie Rose
Cabildo, Arshe Jay
Luzano, Suzanne Sharrie
Salvador, Marianne
Sanchez, Cliff Bradley
Sy, Gian Daniel
Talidong, Cristian
Imperial, Rexchelle Ann

SUBMITTED TO:

Engr. Godofredo T. Avena

June 15, 2026



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



Table of Contents

Project title:	0
Table of Contents:	1
Introduction:	2
Objectives:	2
Scope and Limitations:	2
Methodology:	3
System Design:	6
Technologies Used:	7
Implementation:	7
Testing and Evaluation:	10
Conclusion:	11
Future Work:	11
Appendices:	12
	13
	14



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



I. Introduction

Developed for the CMPE 103 Object-Oriented Programming course, this project presents HARUURROT. This is a Python-based desktop Ride Booking System built using Tkinter to simulate a mobile ride-hailing platform. The application allows users to manage accounts, book rides, view trip histories, cancel reservations, and handle distinct vehicle tiers in an organized manner.

Our primary objective was to apply core Object-Oriented Programming (OOP) principles, specifically inheritance, polymorphism, encapsulation, and abstraction, alongside native file handling for data persistence. This project demonstrates structured software design principles by translating real-world transportation entities into a functional, lightweight desktop application.

II. Objectives

The general objective of this project is to demonstrate the application of object-oriented programming by developing a functional and straightforward ride-booking system app.

- To learn and implement file handling in Python for saving and retrieving application data.
- To design and develop a graphical user interface using Tkinter.
- To understand and implement basic software engineering principles such as modularity and encapsulation in a practical application.

III. Scope and Limitations

Haruurot ride booking system focuses on core functions of a ride hailing system and implementation of OOP concepts. Though it operates at ease, notable limitations are defined with its current operationalities.



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



Scope:

- **Authentication and Session Management:** Account creation, secure login, and logout functionalities for both Passengers and Riders (Drivers).
- **Passenger Features:** Manual entry for pick-up and drop-off locations.
 - Vehicle tier selection including cars, vans, and motorcycles.
 - Dynamic fare estimation based on distance and vehicle multi-tiers.
 - Trip cancellation tools and a personal booking ledger.
- **Rider Features:**
 - Availability toggle for online or offline status.
 - Job board displaying pending rides matching the rider's specific vehicle type.
 - Job acceptance and real-time status updates from Pending to Assigned, and finally to Completed.
- **Developer Info:** Integrated "About Us" and contact profiles for team contributors.

Limitations:

- **No Geospatial Integration:** Lacks real-time GPS tracking and auto-suggest map searches. Inputs are processed as manual strings and explicit kilometer distances.
- **No Live Payment Gateway:** Transactions are completely simulated, meaning fares serve strictly as cost estimations.
- **Local Data Boundary:** Data persistence is bounded by local JSON flat files. Concurrent database handling is not supported.
- **Fixed Resolution:** The UI layout is strictly optimized for a rigid mobile-aspect canvas of 390x780 pixels.

IV. Methodology

The project followed an Agile Feedback-Driven Iterative Approach. While backend developers established core logic, data storage modules, and class schemas, the Quality Assurance (QA) team continuously evaluated the builds. This tight iteration loop enabled the



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



team to rapidly resolve edge-case exceptions, validate input types, and adjust the UI flow based on immediate performance feedback.

System Planning and Design

The system's modular architecture separates core business logic from visual presentation. System capabilities were divided into independent modules:

- `main.py`: Central controller managing application state, configuration, and screen routing.
- `auth.py`: Authentication logic and SHA-256 password hashing.
- `storage.py`: Read and write operations for data persistence.
- `booking.py`: State verification and transaction logic for trips.
- `vehicle.py`: Polymorphic vehicle classes and math engines for fare estimation.

Object-Oriented Design (OOP)

- **Encapsulation:** Class states and user variables are kept private or protected. System parameters are modified strictly via designated getters, setters, and structural backend methods to prevent accidental mutations.
- **Inheritance:** A base Vehicle superclass defines common traits, such as Base Fare, Per-KM Rate, and calculation routines. Subclasses like Car, Van, and Motorcycle inherit these mechanisms while declaring unique scalar values.
- **Polymorphic Behavior:** The system invokes a uniform `calculate_cost(distance)` method across all vehicles, but behavior varies dynamically at runtime based on the specific subclass instance initialized.
- **Abstraction:** Complex internal components, such as JSON parsing, manual string formatting, and error-trapping routines, are hidden from users behind intuitive, event-driven buttons like "Confirm Booking".

Data Collection and Functional Requirements

Use-case analysis established specific criteria for our two primary actors:



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



1. **Passengers:** Require functional registration, active profile editing, mileage-based ride requests, and immediate pending trip cancellations.
2. **Riders:** Require custom profile attributes like vehicle type and capacity, an online or offline switch, and full control over job assignment transitions.

Testing Methodology

Manual white-box and black-box testing was structured by the QA team, Marianne Salvador and Suzanne Luzano, across five vectors:

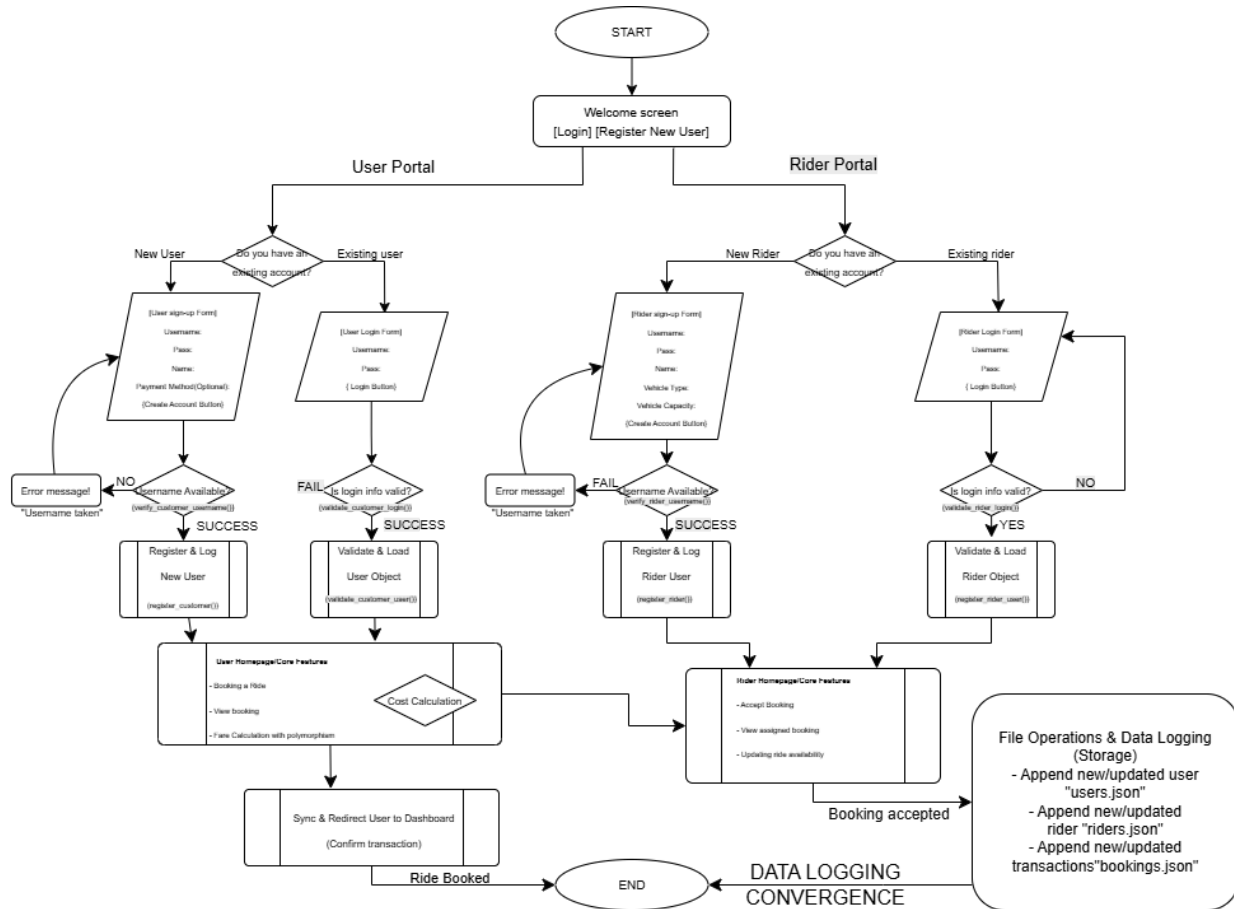
- **Functional Testing:** Verifying success paths for registrations, logins, and live bookings.
- **Input Validation Testing:** Testing edge cases like negative distances (≤ 0), empty strings, duplicated usernames, and mismatched credentials.
- **Role-Based Testing:** Insulating passenger interfaces from rider utilities.
- **State Testing:** Ensuring linear state progression:
Pending → Assigned → Completed or Cancelled
- **File Persistence Testing:** Confirming that local data structures match their respective .json files immediately following an application crash or shutdown event.



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



V. System Design





POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



VI. Technologies Used

The programming language, frameworks, libraries, tools, and platforms used in the project are the following:

Programming Language: Python (latest version)

Tools, Libraries and Platforms:

- Google Docs - Documentation and Planning
- Draw.io - Flowchart
- Git - Collaboration
- Canva/Figma - UI/UX
- VS code - Source code editor

VII. Implementation

HARUURROT operates as a simulated smartphone application using a fixed 390x780 window layout. The HaruurotApp class handles root visual configuration and global session variables: `current_user` (dictionary payload), `current_role` (string flag), and `current_frame` (active UI container). Screen routing utilizes a `.clear()` method that safely de-allocates the active frame wrapper before instantiating the new view, maintaining single-window transitions.

Core Architecture Modules

1. Authentication (auth.py)

Intercepts registration inputs and matches text logins against records within `users.json` or `riders.json`. Passwords are fully hashed using SHA-256 encryption. Rider registration enforces supplementary structural entries for `vehicle_type`, `vehicle_capacity`, and an `is_available` boolean state.

2. Data Storage (storage.py)

Manages stateless, flat-file JSON read and write logic. It exposes three primary utilities:

- `load(path)`: Safely opens a file path and deserializes JSON text into active memory arrays.



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



- `save(path, data)`: Serializes application states back to text using a clean 2-space indentation format.
- `hash_pw(pw)`: Computes the secure SHA-256 string signature for credential security.

3. Vehicle and Fare Engine (vehicle.py)

Implements polymorphic pricing rules based on a standard linear pricing formula:

$$\text{Total Fare} = \text{Base Fare} + (\text{Per-KM Rate}) \times (\text{Distance})$$

A dedicated factory function `get_vehicle(type_name)` matches user choices to their corresponding structural class at runtime without hardcoding explicit imports across individual screen files.

4. Booking Core Logic (booking.py)

Processes transactions and validates system conditions. It controls UUID generation for unique trip references, isolates pending requests per vehicle type for driver job boards, and handles structural safety checks. For example, it verifies that a rider is online before matching, and restricts cancellations strictly to "Pending" trips.

User Interface

Authentication Screens (screens_auth.py)

- **Splash Screen** — Branding screen with Login and Register buttons.
- **Welcome/Login Gateway** — Lets the user choose to log in as a Passenger or Rider.
- **User Login** — Username and password fields with inline error display and a "Forgot Password" placeholder.
- **Register Choice** — Selector for account type (User or Rider), then proceeds to the respective registration form.
- **User Registration** — Collects username, password, full name, and payment method.
- **Rider Gateway** — Intermediary screen before rider login, with a notice that rider registration requires vehicle information.



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



- **Rider Login / Registration** — Similar to user login, but registration additionally collects vehicle type and capacity.

Home and Main Screens (screens_home.py)

Passenger Screens:

- **User Home** — Displays a greeting with the logged-in user's name and menu cards for Book a Ride, View Bookings, and Settings.
- **Book a Ride** — Dropdowns and fields for vehicle type, pickup and drop-off locations, and distance input. Calculates and displays fare dynamically before confirming.
- **View Bookings** — Scrollable list of all the user's bookings, each shown as a card with status badge (Pending, Assigned, Completed, Cancelled) and a Cancel button for eligible bookings.
- **User Settings** — Options to update full name and payment method, with a save function that persists changes to users.json.
- **User Info / About Us** — Displays project credits, team member roles, contributions, and contact details.

Rider Screens:

- **Rider Home** — Displays the rider's name, vehicle type, and availability status, with navigation to Jobs, Assigned Rides, and Settings.
- **Rider Jobs** — Lists all pending bookings that match the rider's vehicle type. Each card shows pickup/drop-off, distance, fare, and an Accept button.
- **Rider Assigned** — Lists all bookings assigned to the logged-in rider, with a Complete button to mark rides as done.
- **Rider Availability** — Toggle switch for the rider's online/offline status, persisted to riders.json.
- **Rider Settings** — Allows the rider to update their full name and view vehicle details.
- **Rider Info / About Us** — Same project credits screen as the user side.



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



Reusable Widget Library (widgets.py)

To maintain consistent design across all screens, a shared widget library was built:

- `header()` — Dark purple header bar with optional back button.
- `scrollable()` — Canvas-based scrollable content area with mousewheel support.
- `field()` — Labeled input with a purple border, placeholder text, and optional password toggle.
- `big_button()` — Wide flat action button.
- `nav_bar()` — Bottom navigation bar with icon and label tabs.
- `menu_card()` — Tappable white card with icon, title, description, and a chevron arrow.

Design System (constants.py)

All colors, fonts, and dimensions are centralized in `constants.py`. The color palette is purple-themed (#3B0E9E dark, #7C3AED accent) with a light gray background (#E8E8F4). Font sizing follows a four-tier hierarchy: Title → H2 → Body → Small

VIII. Testing and Evaluation

Architectural and Validation Findings

Live manual verification proved that modeling system entities as decoupled components minimizes runtime errors. Leveraging encapsulation kept credential records clean, isolating operations to their specific files and preventing memory leakage during state transitions.

By utilizing local file-system handlers via `storage.py`, the application successfully automated updates for records across our independent database files: `users.json`, `riders.json`, and `bookings.json`. Data remained persistent and undamaged through repeated manual shutdown triggers.

Interface and Performance Discussion



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES



Because Tkinter is driven entirely by a single-threaded event loop, all core algorithmic computations, most notably the dynamic distance-fare calculations, were optimized to execute in big O of 1 constant time. This prevents frame-rate stuttering and locks the visual interface to a responsive state during interactive string entries.

The structural boundaries of the system, including the absence of active GPS coordinates and payment checkouts, are not architectural bugs. Instead, they define the baseline parameters of a localized desktop prototype.

IX. Conclusion

The HARUURROT Ride Booking System successfully achieves its academic and architectural objectives. It provides a lightweight, modular desktop application that models a real-world ride-sharing system. By using Python and Tkinter, the project demonstrates how Object-Oriented Programming principles can be applied to real-world scenarios.

Encapsulation ensures secure data handling, inheritance structures our vehicle types cleanly, and polymorphism isolates fare calculations to their specific classes. Additionally, our local JSON configuration provides stable data persistence without the overhead of external database servers. Ultimately, this project highlights how structured software design principles can turn complex real-world logic into a reliable, readable, and highly maintainable application framework.

X. Future Work

While the system fulfills all baseline requirements, future iterations can introduce several scalability and user-experience upgrades:

- **Geospatial Tracking APIs:** Integrating live mapping libraries to replace manual distance inputs with automated route planning and real-time tracking.



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES

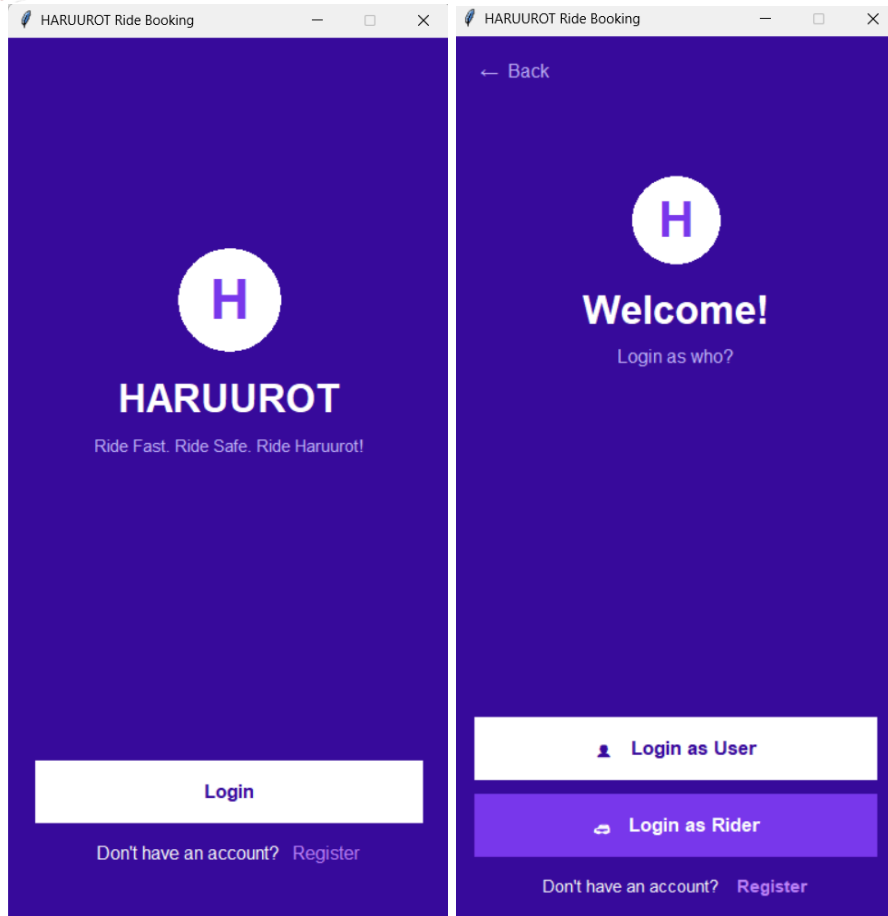


- **External Database Migration:** Moving from flat JSON text files to a relational database like SQLite or PostgreSQL to safely support concurrent operations and advanced data relationships.
- **Live Payment Integration:** Implementing secure sandbox payment gateways such as GCash, PayPal, or Stripe APIs for automated transactional workflows.
- **Dynamic Matching Algorithms:** Enhancing the driver assignment engine with proximity-based matching algorithms to pair passengers with the nearest available rider.

XI. Appendices



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES





POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT
STA. MESA, MANILA, PHILIPPINES

